Benchmark: FastAPI i Flask

porównanie dwóch backendów pod obciążeniem

20.10.2026 20:25

Cel i metoda

Porównujemy dwa popularne frameworki webowe Pythona, wystawiając w każdym te same trzy endpointy. **FastAPI** jest frameworkiem asynchronicznym (standard ASGI) i działa pod serwerem `uvicorn`. **Flask** jest frameworkiem synchronicznym (standard WSGI) i działa pod serwerem `gunicorn`. Wynik zależy od maszyny, na której liczymy, dlatego zestawienie powstaje od nowa przy każdym eksporcie materiałów, razem z opisem sprzętu i wersji bibliotek.

Przebieg pomiaru. Dla każdego backendu: (1) startujemy serwer, (2) rozgrzewamy go krótkim ruchem, (3) generator obciążenia otwiera zadaną liczbę równoległych połączeń. Każde połączenie w pętli wysyła żądanie, czeka na odpowiedź i od razu wysyła następne, i tak przez 5 s. W tym czasie próbkujemy zasoby procesów serwera. Całość powtarzamy dla liczby połączeń równej 1, 10, 50, 200. Backendy mierzymy kolejno, nie jednocześnie, żeby nie konkurowały o procesor.

Co oznaczają metryki.

- **RPS** (zapytania na sekundę): ile zapytań serwer obsłużył w ciągu sekundy. Im więcej, tym lepiej.
- **p95 [ms]**: opóźnienie, w którym mieści się 95% odpowiedzi (jedna na dwadzieścia jest wolniejsza). Mówi o tym, co widzi użytkownik, lepiej niż średnia. Im mniej, tym lepiej.
- **CPU [%]**: czas procesora zużyty przez procesy serwera podzielony przez czas pomiaru. 100% oznacza pełne obciążenie jednego rdzenia. Wartość może lekko przekroczyć 100%, bo część pracy (np. obsługa sieci w jądrze systemu) wykonuje się poza interpreterem Pythona i może biec równolegle.
- **CPU [ms/zap.]**: ile czasu procesora kosztuje jedno zapytanie. Pokazuje efektywność, niezależnie od tego, ile zapytań przyszło.
- **RAM [MB]**: pamięć zajmowana przez procesy serwera (miara PSS, która poprawnie dzieli pamięć współdzieloną między procesy).

Dlaczego opóźnienia rosną razem z liczbą połączeń. Gdy serwer jest nasycony, czyli nie nadąża z obsługą, nowe żądania czekają w kolejce. Zgodnie z prawem Little'a liczba żądań w systemie jest równa przepustowości razy czas odpowiedzi, czyli

$$\text{czas odpowiedzi} \approx \frac{\text{liczba połączeń}}{\text{przepustowość}}.$$

Jeśli przepustowość jest stała (serwer wykorzystał już cały dostępny procesor), to dziesięć razy więcej połączeń daje około dziesięć razy dłuższy czas odpowiedzi. Dlatego na wykresach przepustowość „wypłaszcza się", a opóźnienie rośnie prawie liniowo.

Jak czytać wykresy. Każdy scenariusz ma jeden wykres z czterema panelami: przepustowość (wyżej znaczy lepiej), opóźnienie p95 (niżej znaczy lepiej; oś jest logarytmiczna, więc równe odstępy oznaczają stałe krotności), procesor i pamięć. Oś pozioma pokazuje liczbę równoległych połączeń, również w skali logarytmicznej.

Ograniczenia. Generator obciążenia i serwer działają na tej samej maszynie i dzielą procesor. Każdy serwer ma jeden proces. Wynik to pojedynczy przebieg, więc drobne różnice (kilka procent) mieszczą się w szumie pomiarowym.

Konfiguracja pomiaru

Procesor	**12th Gen Intel(R) Core(TM) i7-12700F**
Rdzenie	12 fizycznych, 20 logicznych
Pamięć RAM	31,2 GB
System	Linux-6.17.0-40-generic-x86_64-with-glibc2.39
Python	3.12.15
FastAPI	uvicorn 0.54.0 (pętla: asyncio, parser HTTP: h11), 1 proces, endpointy async
Flask	gunicorn 26.2.0, klasa gthread, 1 proces, 16 wątków, endpointy synchroniczne
Biblioteki	fastapi 0.142.2, pydantic 2.13.5, flask 3.1.3, werkzeug 3.1.9, aiohttp 3.14.4
Generator obciążenia	aiohttp, 4 procesów klienta; serwer i klient na tej samej maszynie, backendy mierzone kolejno
Pomiar punktu	5 s po 1 s rozgrzewki; równoległe połączenia: 1, 10, 50, 200
Błędy (łącznie)	FastAPI: 0; Flask: 0

Scenariusze

GET /ping

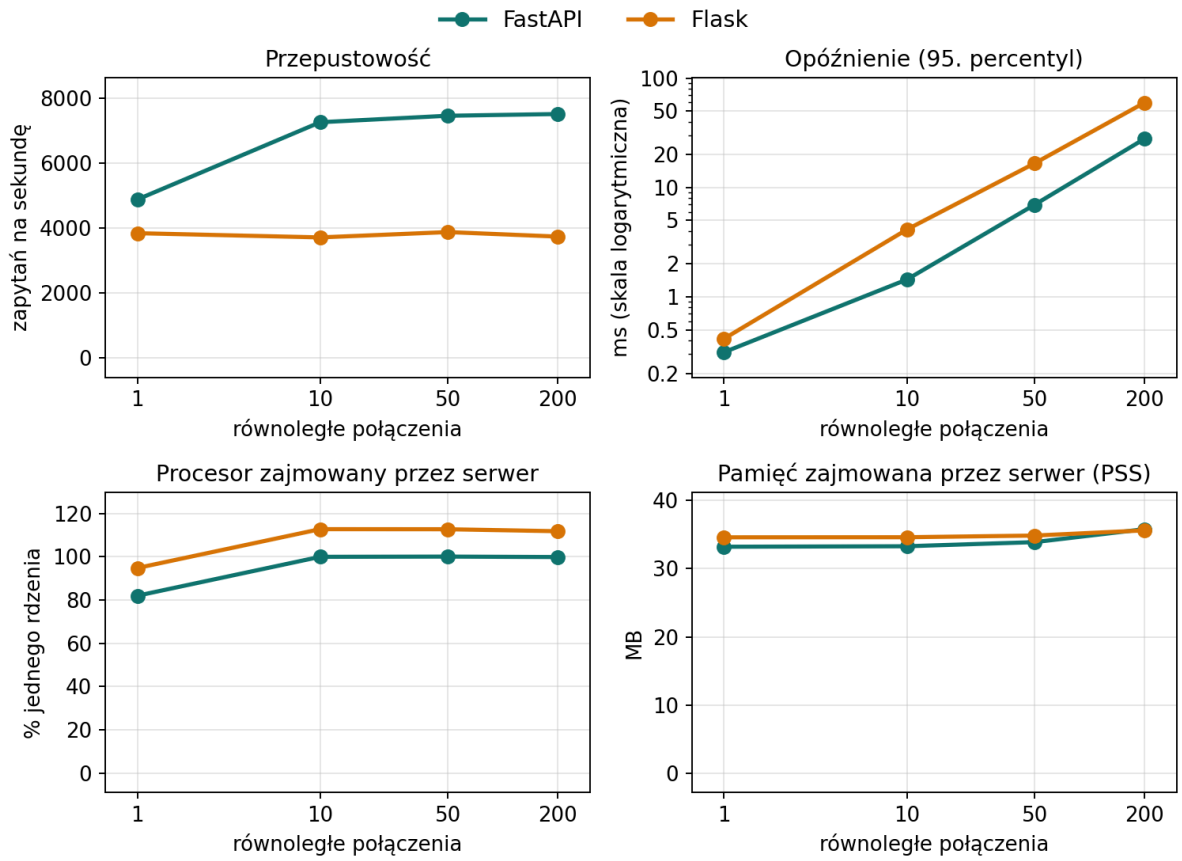
Co robi endpoint. Zwraca stały, mały obiekt JSON (`{"status": "ok"}`) i nie wykonuje żadnej pracy po stronie aplikacji. Mierzymy więc wyłącznie narzut frameworka i serwera: odczyt żądania HTTP, wywołanie funkcji i zapis odpowiedzi.

Czego się spodziewać. Praca na jedno zapytanie jest krótka, ale w całości zajmuje procesor. Każdy serwer działa w jednym procesie Pythona, więc wykorzysta najwyżej jeden rdzeń. Gdy rdzeń jest zajęty w 100%, dokładanie połączeń nie podnosi przepustowości, tylko wydłuża kolejkę. Wygrywa ten serwer, który na jedno zapytanie zużywa mniej czasu procesora.

Co pokazały pomiary. Największą przepustowość FastAPI osiągnął przy 200 połączeniach: 7519 zapytań/s. Flask osiągnął 3879 zapytań/s przy 50 połączeniach, więc FastAPI obsłużył 1,9 razy więcej zapytań. Przy 200 połączeniach 95% odpowiedzi FastAPI mieściło się w 28 ms, a Flaska w 59,9 ms (FastAPI odpowiadał 2,1 razy szybciej). W tym czasie serwery zajmowały 100% i 112% jednego rdzenia, a jedno zapytanie kosztowało średnio 0,13 ms procesora w

FastAPI i 0,3 ms we Flasku. Pamięć serwerów zmieniła się w trakcie całego scenariusza najwyżej o 2,6 MB.

Sprawdzenie prawem Little'a: przy 200 połączeniach FastAPI miał przepustowość 7519 zapytań/s, co daje 26,6 ms na odpowiedź (zmierzona mediana: 26,3 ms). Dla Flaska wychodzi 53,5 ms (mediana: 51,9 ms).



Rysunek 1: Wyniki scenariusza GET /ping: przepustowość, opóźnienie, procesor i pamięć w funkcji liczby połączeń.

Połącz.	FastAPI					Flask				
	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]
1	4884	0,3	82	0,17	33,2	3845	0,4	95	0,25	34,6
10	7266	1,4	100	0,14	33,3	3715	4,1	113	0,3	34,6
50	7466	7	100	0,13	33,9	3879	16,7	113	0,29	34,9
200	7519	28	100	0,13	35,8	3740	59,9	112	0,3	35,6

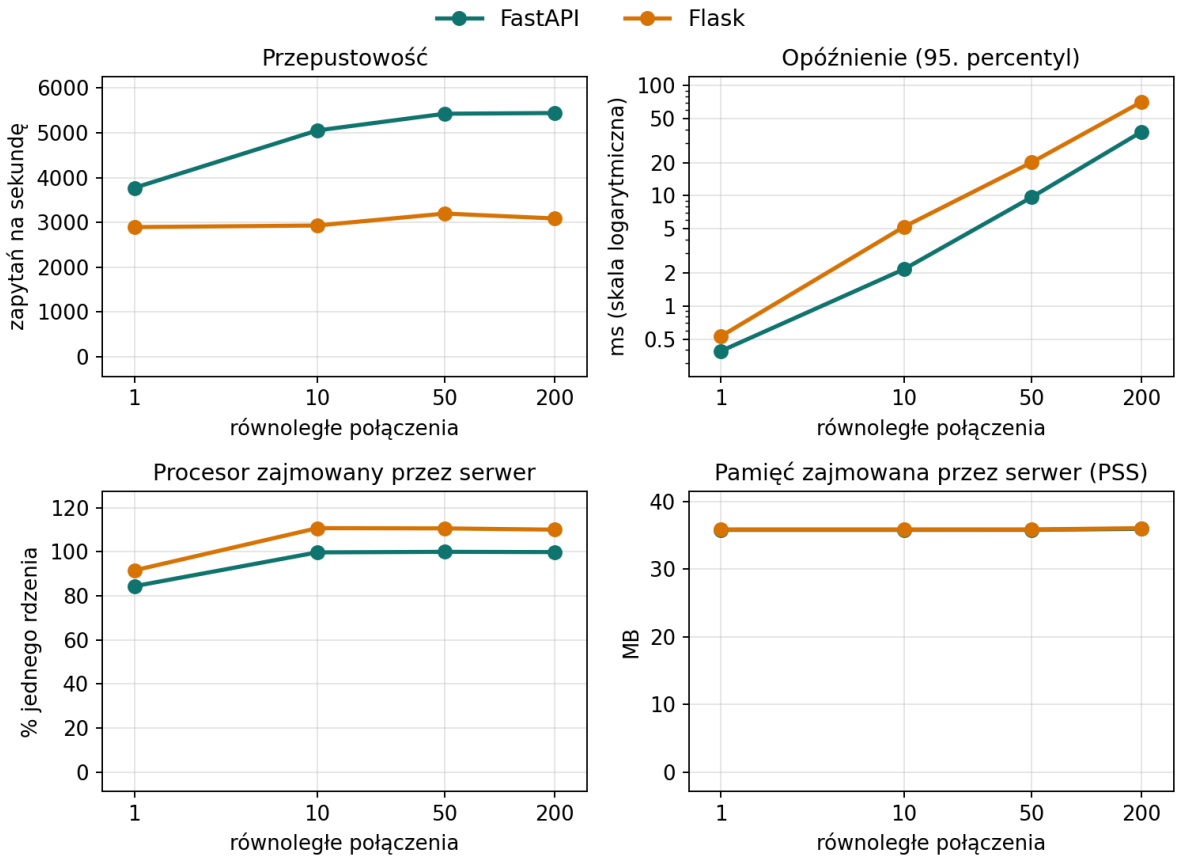
POST /suma (100 liczb, walidacja)

Co robi endpoint. Przyjmuje w treści żądania JSON z listą stu liczb (`{"liczby": [...]}`), sprawdza, czy dane mają poprawny kształt, i zwraca ich sumę oraz liczbę. Pracy jest więcej niż w `/ping` : serwer musi odczytać treść, zamienić JSON na obiekty Pythona i zwalidować każdy element.

Czego się spodziewać. We FastAPI walidację robi `pydantic` (jego rdzeń jest napisany w Rust), a we Flasku robimy ją ręcznie, pętlą w Pythonie. Obie wersje sprawdzają to samo, ale koszt procesora na zapytanie jest inny. Scenariusz nadal jest ograniczony procesorem, więc obowiązują te same zasady co w poprzednim: po nasyceniu rdzenia przepustowość stoi, a opóźnienie rośnie wraz z liczbą połączeń.

Co pokazały pomiary. Największą przepustowość FastAPI osiągnął przy 200 połączeniach: 5439 zapytań/s. Flask osiągnął 3197 zapytań/s przy 50 połączeniach, więc FastAPI obsłużył 1,7 razy więcej zapytań. Przy 200 połączeniach 95% odpowiedzi FastAPI mieściło się w 38 ms, a Flaska w 71,1 ms (FastAPI odpowiadał 1,9 razy szybciej). W tym czasie serwery zajmowały 100% i 110% jednego rdzenia, a jedno zapytanie kosztowało średnio 0,18 ms procesora w FastAPI i 0,36 ms we Flasku. Pamięć serwerów zmieniła się w trakcie całego scenariusza najwyżej o 0,2 MB.

W porównaniu ze scenariuszem `/ping` maksymalna przepustowość FastAPI spadła do 72%, a Flaska do 82% wartości z `/ping`. Różnica to koszt odczytania i sprawdzenia treści żądania.



Rysunek 2: Wyniki scenariusza POST /suma (100 liczb, walidacja): przepustowość, opóźnienie, procesor i pamięć w funkcji liczby połączeń.

Połącz.	FastAPI					Flask				
	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]
1	3771	0,4	84	0,22	35,8	2895	0,5	92	0,32	35,9
10	5050	2,1	100	0,2	35,8	2930	5,2	111	0,38	35,9

Połącz.	FastAPI					Flask				
	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]
50	5424	9,7	100	0,18	35,8	3197	20	111	0,35	35,9
200	5439	38	100	0,18	36	3088	71,1	110	0,36	36,1

GET /czekaj (50 ms oczekiwania)

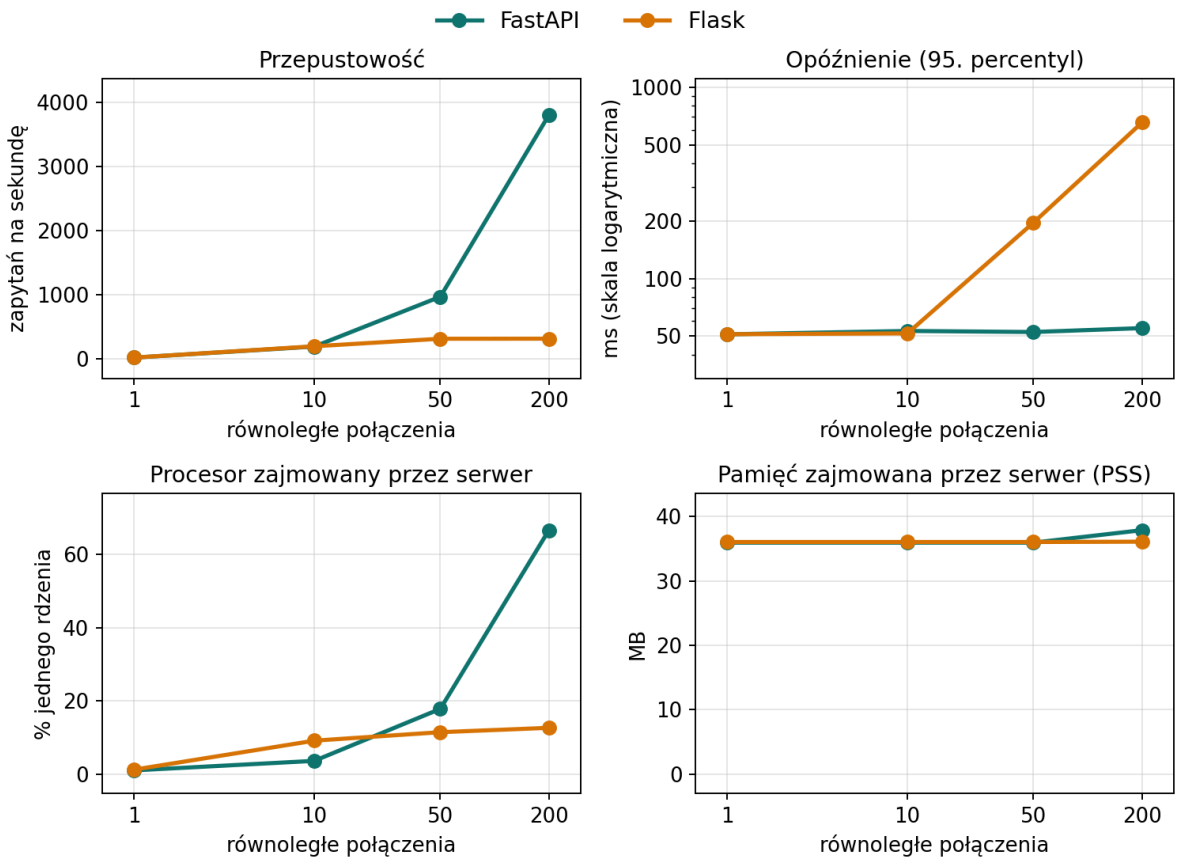
Co robi endpoint. Czeka 50 ms, po czym zwraca stały JSON. To odpowiednik zapytania do bazy danych albo zewnętrznej usługi: aplikacja nic nie liczy, tylko czeka na odpowiedź. Procesor prawie nie pracuje, więc ograniczeniem nie jest jego moc, tylko to, **ile żądań serwer potrafi mieć w toku jednocześnie**.

Dlaczego frameworki zachowują się tu różnie. FastAPI jest asynchroniczny: funkcja `async def` z `await asyncio.sleep(...)` na czas czekania oddaje sterowanie pętli zdarzeń, która w tym czasie obsługuje inne żądania. Jeden proces może więc czekać równolegle na setki zapytań. Flask jest synchroniczny: `time.sleep(...)` blokuje wątek na cały czas czekania. Skonfigurowany serwer ma 16 wątków, więc obsłuży jednocześnie najwyżej tyle żądań, a pozostałe czekają w kolejce.

Co mówi teoria. Przepustowość nie może przekroczyć liczby żądań w toku podzielonej przez czas jednego żądania. Dla Flaska to 16 / 0,05 s = 320 zapytań/s niezależnie od liczby połączeń. Dla FastAPI przy 200 połączeniach limit to 200 / 0,05 s = 4000 zapytań/s i rośnie wraz z liczbą połączeń.

Co pokazały pomiary. Największą przepustowość FastAPI osiągnął przy 200 połączeniach: 3798 zapytań/s. Flask osiągnął 315 zapytań/s przy 200 połączeniach, więc FastAPI obsłużył 12,1 razy więcej zapytań. Przy 200 połączeniach 95% odpowiedzi FastAPI mieściło się w 55 ms, a Flaska w 654,4 ms (FastAPI odpowiadał 11,9 razy szybciej). W tym czasie serwery zajmowały 67% i 13% jednego rdzenia, a jedno zapytanie kosztowało średnio 0,18 ms procesora w FastAPI i 0,4 ms we Flasku. Pamięć serwerów zmieniła się w trakcie całego scenariusza najwyżej o 1,9 MB.

Zmierzona przepustowość przy 200 połączeniach to 95% limitu teoretycznego dla FastAPI i 98% limitu dla Flaska. Przy 1 połączeniu każdy serwer ma w toku jedno żądanie naraz, więc przepustowość wynosi 20 zapytań/s dla FastAPI i 20 dla Flaska, czyli około 20 (jedno żądanie na 50 ms).



Rysunek 3: Wyniki scenariusza GET /czekaj (50 ms oczekiwania): przepustowość, opóźnienie, procesor i pamięć w funkcji liczby połączeń.

Połącz.	**FastAPI**					**Flask**				
	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]	RPS	p95 [ms]	CPU [%]	CPU [ms/ zap.]	RAM [MB]
1	20	51	1	0,51	36	20	51,1	1	0,61	36,1
10	192	53,2	4	0,19	36	196	51,6	9	0,46	36,1
50	968	52,5	18	0,18	36	314	196	11	0,36	36,1
200	3798	55	67	0,18	37,9	315	654,4	13	0,4	36,1